# Cartographer Relocalization Navigation

**Cartographer Relocalization Navigation**

1. Course Content
2. Principle Introduction
4. Running the Cases
    4.1 Preparation
    4.2 Starting the Navigation Node
5. Source Code Analysis
    5.1 Viewing the TF Tree

## 1. Course Content

1. Run the example program; the robot can quickly localize its position in the map near its initial position and perform navigation

## 2. Principle Introduction

- The default localization algorithm of Navigation 2 is AMCL (Adaptive Monte Carlo Localization). Here the default AMCL localization method is replaced with cartographer's localization method, while other navigation functions remain unchanged.

## 4. Running the Cases

### 4.1 Preparation

- A grid map and a pose map in pbstream format are required. To build a pose map in pbstream format, refer to the [Cartographer Algorithm Mapping] chapter

> **Caution**
>
> Note that the grid map and the pose map in pbstream format must correspond to each other.

### 4.2 Starting the Navigation Node

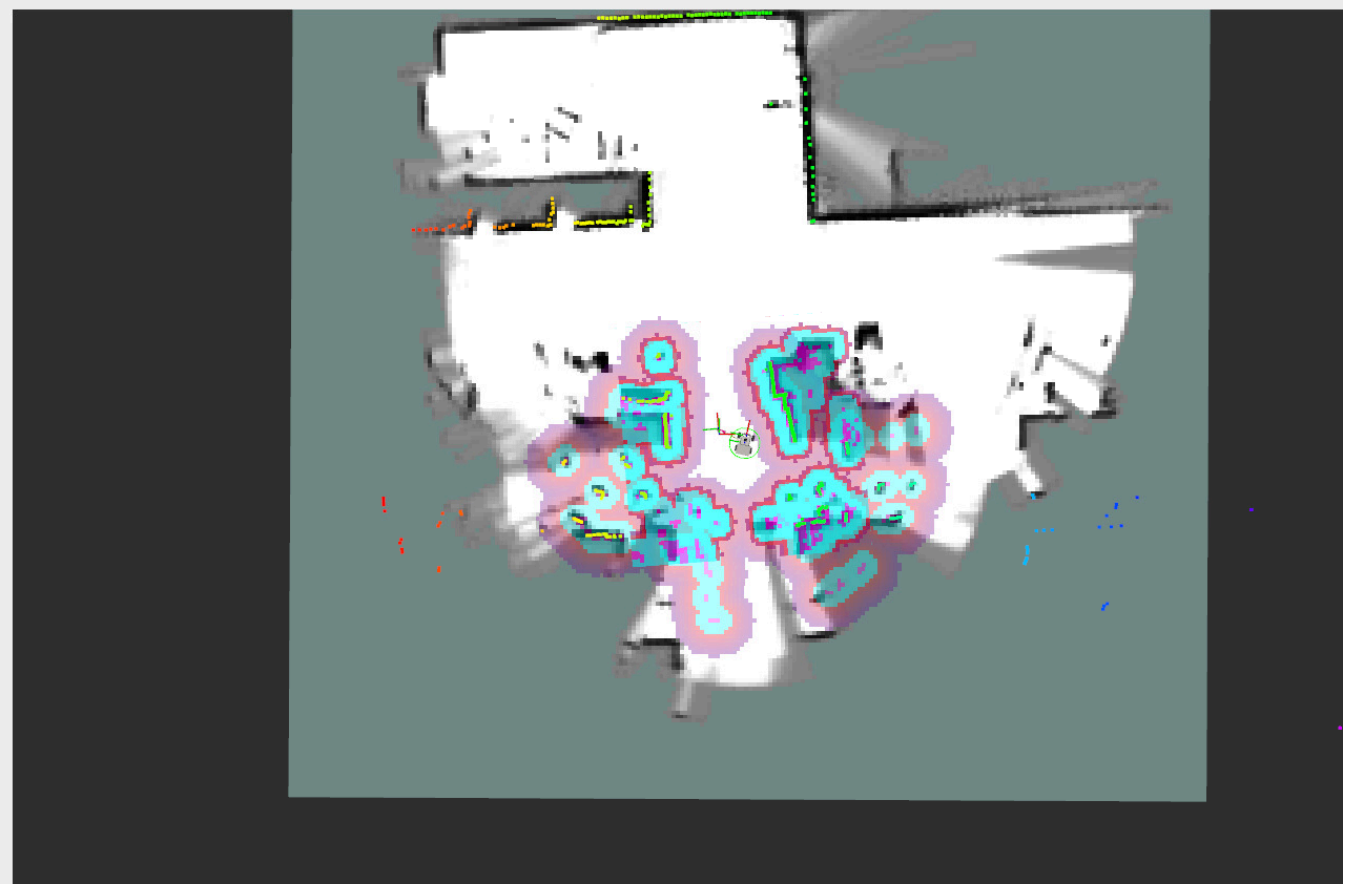
```
ros2 launch yahboomcar_bringup nav2.launch.py localization_mode:=cartographer
```

> **Tip**
>
> **Launch parameter description**
>
> - localization_mode
>     - Global localization method; here we choose to use cartographer's localization method
> - load_state_filename
>     - The name of the pbstream pose map; if cartographer is selected as the localization method

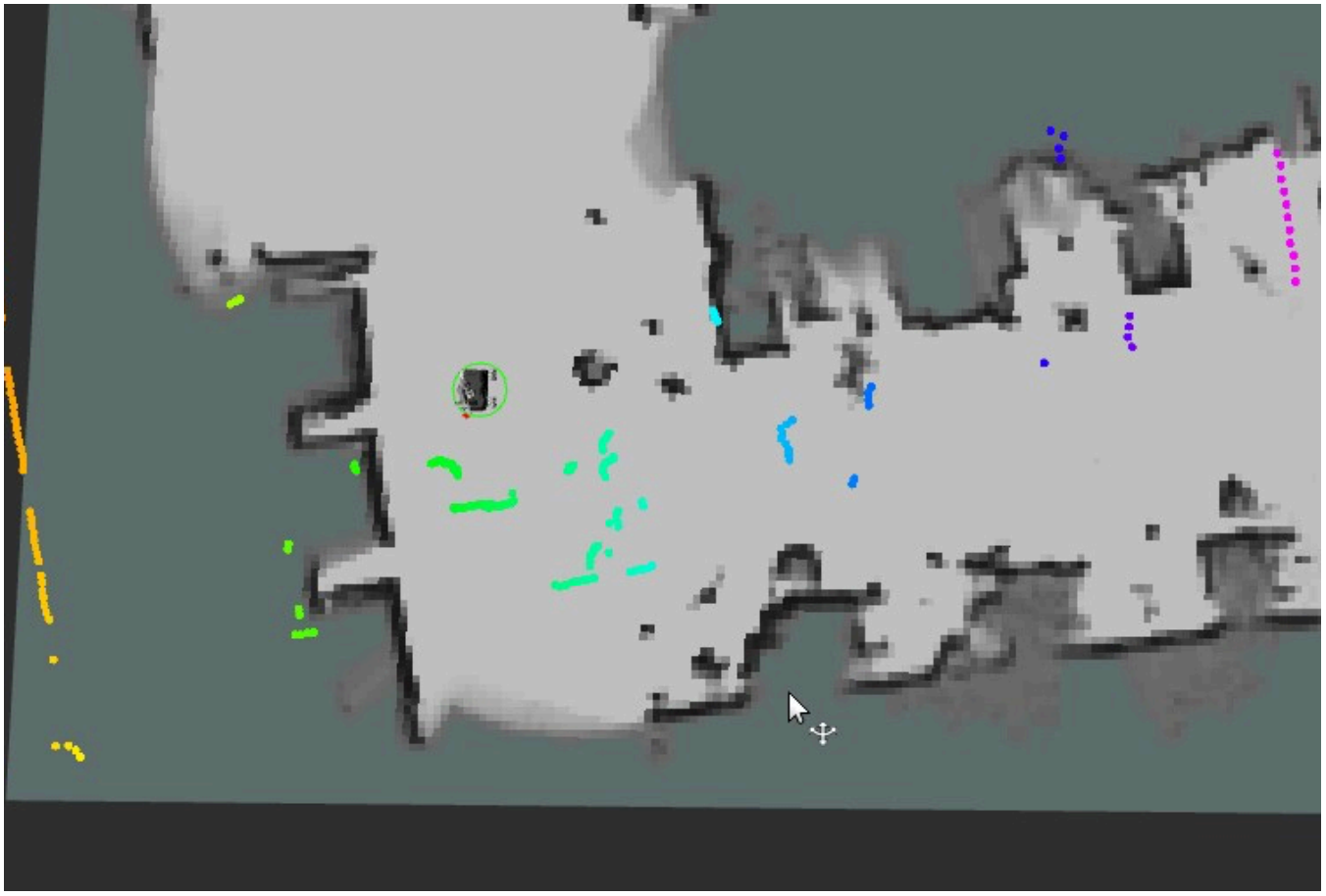
during navigation, the pose map name must be passed in; default value map.pbstream

- map
    - The map name passed in; the default value is map.yaml

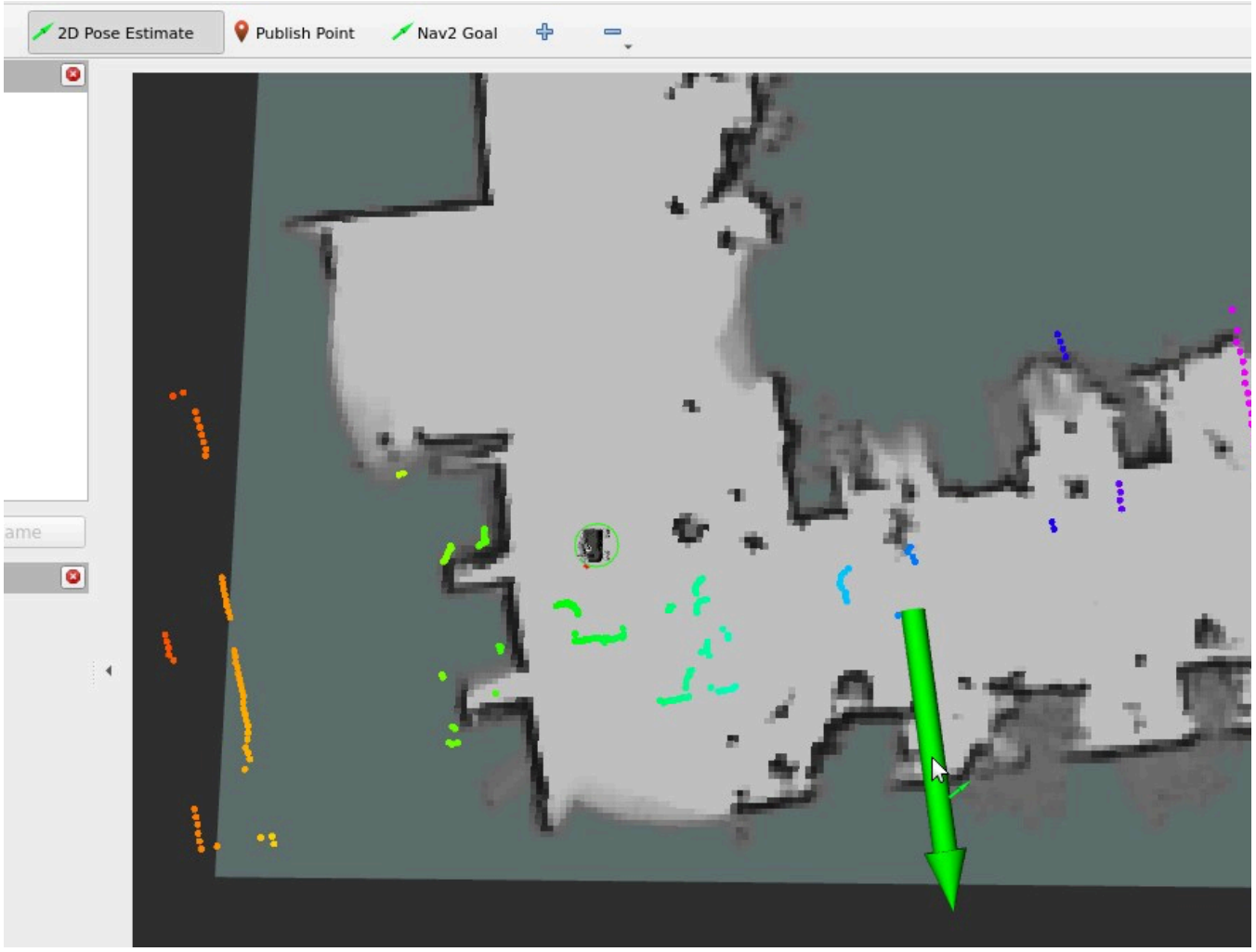
- From rviz you can see that the robot can automatically complete the initial position estimation without manually initializing the position; the remaining navigation steps are the same as when using the AMCL localization method normally.



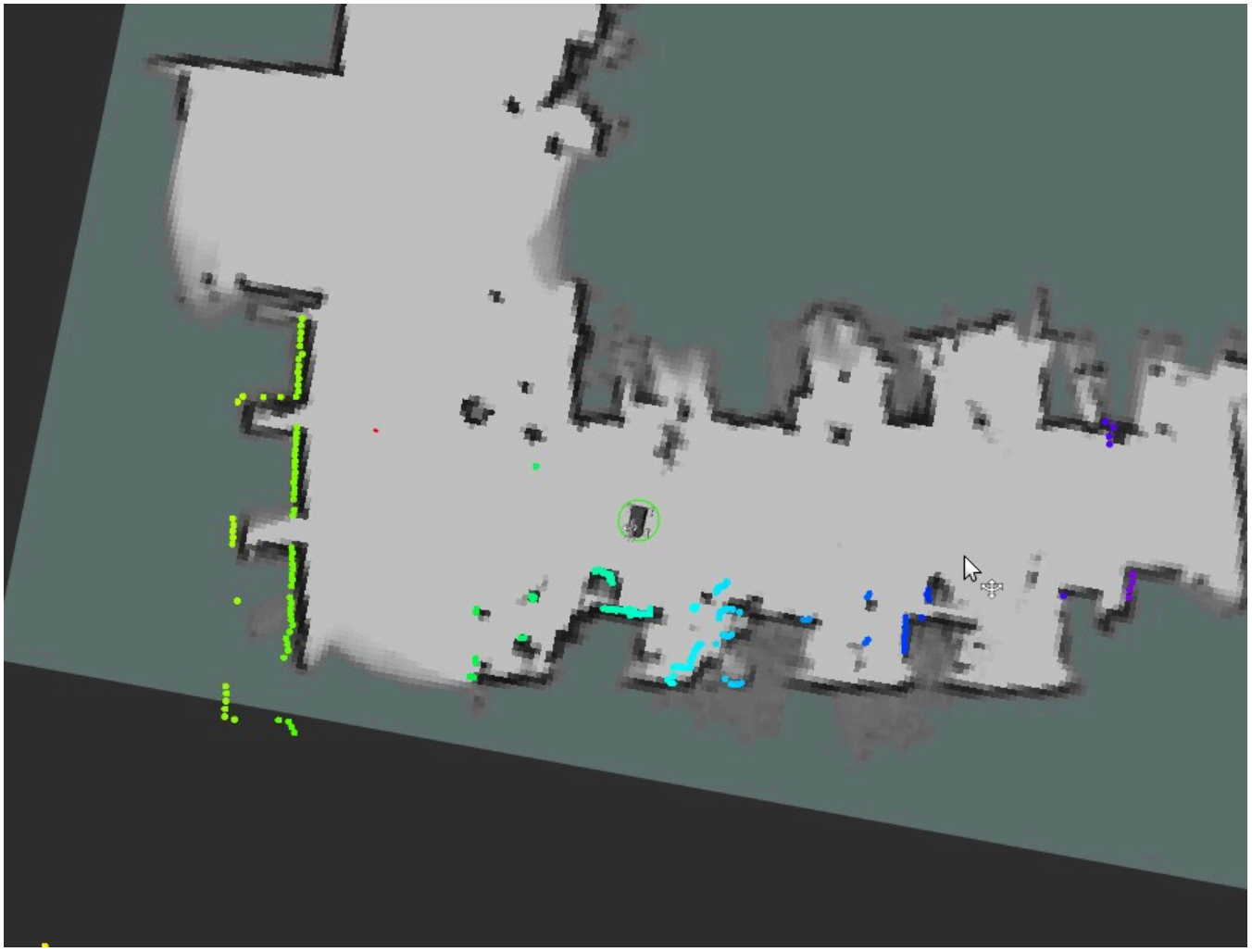
- Relocalization feature: if the car is not at the initial position or loses global localization due to reasons such as being moved manually, as shown below, the laser no longer matches the environmental contours



- Use the 【2D Pose Estimate】 tool to publish the car's approximate position; it will automatically perform matching localization from the pose graph to find the precise position




- A high degree of agreement between the LiDAR and the environment indicates that the global localization is accurate and the relocalization is successful



**Important**

- Due to body occlusion on the M1PRO, the effective LiDAR range is only about 180 degrees. In scenes with degraded or similar environmental features, relocalization has a small probability of failing to solve. You can try adding feature objects at the edges of the environment and moving back and forth multiple times during the mapping stage to increase the number of submaps;
- The relocalization feature in the next chapter, [slam_toolbox Localization Navigation], is similar to this section; the difference is the localization method

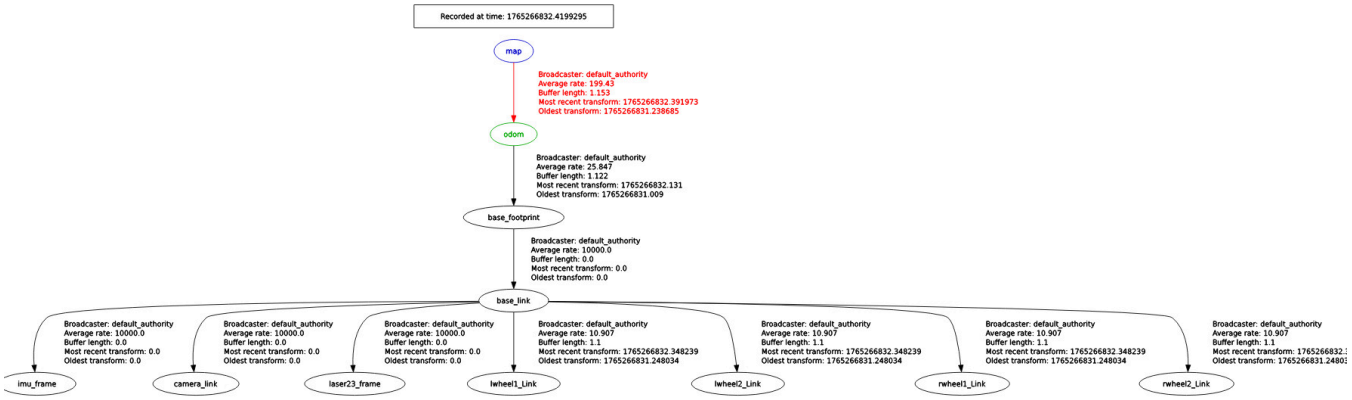
# 5. Source Code Analysis

## 5.1 Viewing the TF Tree

Input in the terminal:

```
ros2 run rqt_tf_tree rqt_tf_tree
```

- Here, the TF transform from the map coordinate frame to the odom coordinate frame in the TF tree is provided by cartographer.




- launch file path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/nav2.launch.py
```

- The key program snippet is as follows, which starts the navigation2 navigation stack nodes and the cartographer pure localization node

```
    #Cartographer positioning navigation
    cartographer_localization_nav = GroupAction(
        condition=IfCondition(PythonExpression(["'", localization_mode, "' == 'cartographer' 
and '", slam, "' == 'False' "])),
        actions=[
            #Start navigation2 navigation file (no global positioning)
            PushRosNamespace(namespace=namespace),
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(nav2_launch_mod),
                launch_arguments={
                                "map": map_dir,
                                "params_file": params_file,
                                "use_sim_time":use_sim_time,
                                }.items()),
        Node(
            package =   'cartographer_ros',
            executable = 'cartographer_node',
            parameters = [{'use_sim_time': False}],
            arguments = [
                        '-configuration_directory', configuration_directory,
                        '-configuration_basename', 'cartographer_2d.lua',
                        '-load_state_filename', load_state_filename],
            remappings = [('imu', '/imu/data_raw')],
            output = 'screen'
            ),
        Node(
            package = 'cartographer_ros',
            executable = 'cartographer_occupancy_grid_node',
            parameters = [
```

```
                {'use_sim_time': False},
                {'resolution': 0.05}],
            ),
            #Log Printing
            LogInfo(msg=[Fore.GREEN+'====================Robot-M3 Navigation Start with
cartographer localization====================',Fore.RESET])
        ]
    )
```